Our network also has a weight value for the connections, which indicates the strength of connections. In our case, our edge list represents how many times the source favorited the tweets of the target. Sam favorited one tweet made by Pris, and Roy favorited 5 tweets made by Pris, and so on. This is a *weighted network* because the edges have a value. It's a *directed network* because the edges have direction. Therefore, we have a *weighted directed network*, and we need to account for both weight and direction in our network visualizations.

Technically, you only need an edge list to create a network, because you can derive a list of nodes from the unique values in the edge list. This is done by traditional network analysis software packages like Gephi. Although you can derive a node list with JavaScript, it's more common to have a corresponding node list that provides more information about the nodes in your network, like we have in the following listing.

Listing 6.2 nodelist.csv

```
id,followers,following
sam,17,500
roy,83,80
pris,904,15
tully,7,5
kim,11,50
mo,80,85
pat,150,300
lee,38,7
al,12,12
```

Because these are Twitter users, we have more information about them based on their Twitter stats, in this case, the number of followers and the number of people they follow. As with the edge list, it's not necessary to have more than an ID. But having access to more data gives you the chance to modify your network visualization to reflect the node attributes.

How you represent a network depends on its size and the nature of the network. If a network doesn't represent discrete connections between similar things, but rather the flow of goods or information or traffic, then you could use a Sankey diagram like we did in chapter 5. Recall that the data format for the Sankey is exactly the same as what we have here: a table of nodes and a table of edges. The Sankey diagram is only suitable for specific kinds of network data. Other chart types, such as an adjacency matrix, are more generically useful for network data.

Before we get started with code to create a network visualizations, let's put together a CSS page so that we can set color based on class and use inline styles as little as possible. Listing 6.3 gives the CSS necessary for all the examples in this chapter. Keep in mind that we'll still need to set some inline styles when we want the numerical value of an attribute to relate to the data bound to that graphical element, for example, when we base the stroke-width of a line on the strength of that line.